Relational Database Basics

(https://github.com/learn-co-curriculum/python-p3-relational-database-basics) (https://github.com/learn-co-curriculum/python-p3-relational-database-basics/issues/new)

Learning Goals

- Use primary and foreign keys to create the relations in a relational database.

Key Vocab

- **Primary Key**: a number that uniquely identifies one record in a table.
- **Foreign Key**: a column or group of columns that connects one table to another.
- **Join**: a query that returns related records from multiple tables in a single record.
- **One-to-Many**: a type of relationship between tables where one record in table A is connected to multiple records in table B. **e.g.** One person ordering multiple drinks at a bar.
- **Many-to-Many**: a type of relationship between tables where multiple records in table A are connected to multiple records in table B. **e.g.** Students have many classes and classes have many students.

Introduction

We'll introduce the concept of relational databases and how they recognize relations among stored items of information.

Relational Databases

Let's say that you've been hired by a big and important company to do the payroll for all of their employees. We'll call it MyFace (inspired by nothing in particular). Every two weeks, you need to look up each and every employee and how much they get paid, and send them a check *and* send a notice of that check to their manager (managers, after all, should know when their employees are getting paid).

In addition, let's say that managers get paid every month, instead of every two weeks. So, once a month we need to go through the spreadsheet again, find *just the managers*, and send them *their* checks. In such a situation, we would need a place to store all of the managers and employees.

Using a spreadsheet, your storage system might look something like this:

So every two weeks, we would have to look through every single entry in this spreadsheet, send each person their check, and then figure out a way to identify an employee's manager to send that manager a confirmation that each employee has been paid. We need some way to *associate* the employees to their manager.

We could add a "Manager" column to the spreadsheet that would be filled out with the name of that person's manager (if that person is an employee and not a manager themselves). This is getting messy. Not only do we have to do a lot of searching through the spreadsheet and manual detection of who is an employee and who is a manager, but we also have to match each employee with the name of their manager. If only there was some way to simplify our system!

Enter relational databases. A relational database, simply put, is **a database structured to recognize relations among stored items.** In such a system, it would be easy to tell an employee that they *belong to* a certain manager and to tell a manager that they *have many* employees.

Relational Database Structure

Relational databases have one unique and very powerful feature that allows us to establish relationships between multiple tables: **primary keys** and **foreign keys**.

- **Primary Key**: a column in a table with an identifier (ID) that uniquely identifies one specific record, or row, in a table
- **Foreign Key**: a column in one table that refers to a primary key in another table

Continuing with our payroll example from earlier, employees and managers would be stored in their own **tables**. A table is like a spreadsheet; it has columns and rows.

Our managers table would look something like this:

And our employees table would look something like this:

Our employees table has a "Manager ID" column, filled with the ID number of that person's manager. In a relational database, every row has a number, called a **primary key**. Relationships between tables can be established by using a **foreign key** column, like our "Manager ID" column, that uses that primary key of another table to refer to a member of that table.

In the images above, you can see that in the employees table Bob and Karen both have a value of 1 in the Manager ID column (**foreign key**). How can we tell who their manager is? We need to look to the other table for that information — the managers, and find the manager with the Manager ID (**primary key**) of 1. That exact process of using a key in one table to identify a corresponding row in another table is what SQL will do for us programatically when we give it the right instructions!

Why should our foreign key, our point of reference between an employee and his or her manager, be a number? Why not just use the manager's name? Well, names are very rarely unique. What if MyFace hires a new manager, also named Steve? It's a popular name, after all. How would our database know *which* Steve manages which employees. Primary keys, on the other hand, **are always unique!**

Now, with these separated but related tables, our job just got a lot easier. We should thank...

Edgar Codd

Edgar Codd invented the concept of the relational database, in other words, he came up with the idea that storing data in tables, indexed by primary key and related by foreign keys would *normalize* that data.

> At the time, Nixon was normalizing relations with China. I figured that if he could normalize relations, then so could I.
>
> — Edgar Codd

Codd developed Relational Database Theory as a graduate student. Afterwards, he worked with Don Chamberlain at IBM to create a language that would allow the user to traverse these relational databases for specific subsets of information.

The language they created was SQL—Structured (or Standard) Query Language. SQL allows the user to carry out queries like "find the employees who make more than the managers", or "find the managers whose employees make under $X" in an efficient and sensical manner. Before SQL, database queries were all about *where* data was stored, instead of *what* data a user is looking for.

Conclusion

Working with relational databases takes a bit more abstract thought than working with a flat spreadsheet. Instead of visualizing the data all in one place, we have to consider how data in multiple places can be related. Thankfully, in SQL, the only way to relate between two different tables is using **foreign keys** and **primary keys**, so once you are comfortable with this concept, you'll be well on your way to mastering relational databases.

Resources

- **What is a Relational Database? - Google Cloud (https://cloud.google.com/learn/what-is-a-relational-database)**
- **Difference between Primary Key and Foreign Key - GeeksforGeeks (https://www.geeksforgeeks.org/difference-between-primary-key-and-foreign-key/)**